



The Lazy Developer

15

MODULE

UI, UX, and a mobile first approach

In most cases, the vast majority of users will be mobile viewers. Your site needs to be able to easily support this and with a quick and efficient manner. Catering to both desktop and mobile users with different layouts is key.

Table of Contents

- 01. Think like a user (20 min)
- 02. Get others to give you initial feedback (20 min)
- 03. Make it responsive with the latest standards (22 min)
- 04. Try your hand at Figma (18 min)
- 05. Come up with scenarios to test features (22 min)

Think Like a User

Imagine you walk into a brand-new grocery store for the first time. The entrance leads you straight into the frozen food aisle. There are no signs on the ceiling. The checkout counters are hidden behind the dairy section. You can't find the bread, the staff are nowhere to be seen, and by the time you locate the exit, you've already decided you're never coming back.

That's exactly what happens when someone visits a poorly designed app or website. They land on the page, can't figure out where to go, get frustrated, and leave. The store might have had the best produce in town, but none of that matters because the experience was terrible.

This is why thinking like a user is the single most important skill you can develop as someone building applications. It doesn't matter how clever your code is or how beautiful your colour palette looks. If people can't figure out how to use your app within the first few seconds, they're gone.

Why User Empathy Matters More Than You Think

Here's something that surprises a lot of first-time builders: you are not your user. You know every page, every button, every feature of your app because you built it. You know that the settings page is tucked behind that little gear icon in the bottom-left corner. You know that you need to scroll past the hero section to find the pricing. But your users don't know any of that. They're seeing everything for the first time, often on a phone, often distracted, and often with one thumb while waiting for a bus.

User empathy means deliberately stepping outside your own perspective and experiencing your app as if you've never seen it before. It means asking questions like:

- If I landed on this page right now with no context, would I know what this app does?
- Can I find the most important action (sign up, buy, contact) within 5 seconds?
- Does anything confuse me or make me hesitate?
- Would my parents be able to use this?

That last question is surprisingly powerful. If you can imagine your least tech-savvy family member navigating your app successfully, you're probably on the right track.

Common UX Mistakes Beginners Make

When you're building your first few apps, certain mistakes show up over and over again. Knowing about them upfront saves you from having to fix them later (which is always harder).

Mistake	Why It Happens	How to Fix It
Too much text on one page	You want to explain everything	Break content into sections, use headings, keep paragraphs short

Unclear call-to-action	You have multiple buttons competing for attention	One primary action per page, make it visually dominant
Confusing navigation	You added pages as you built, without planning	Plan your navigation structure before building
Tiny tap targets on mobile	You designed on a big monitor	Buttons should be at least 44x44 pixels for touch screens
No visual hierarchy	Everything looks the same size and weight	Use font sizes, bold text, and colour to guide the eye
Inconsistent styling	You changed your mind halfway through	Pick a design system (like shadcn/ui) and stick with it
Hidden important information	You buried key details below the fold	Put the most critical information where users see it first
No feedback on actions	User clicks a button and nothing visible happens	Show loading states, success messages, error messages
Not using the Core Web Vitals Test	The Google Search Console will do a thorough test on UI, UX, SEO and performance for both mobile and desktop!	

Here's a prompt you could use right now to catch these issues in your own project:

```
Review my app's homepage component and identify any common UX mistakes. Look for: unclear calls-to-action, poor visual hierarchy, too much text, confusing navigation, missing feedback states, and accessibility issues. Suggest specific fixes for each problem you find.
```

Navigation Patterns That Work

Navigation is the skeleton of your app. Get it wrong and everything feels broken. Get it right and users don't even notice it, which is exactly the point.

There are a few navigation patterns that have been battle-tested across millions of websites and apps:

Top Navigation Bar (Navbar)

The most common pattern for websites. Your logo on the left, main links in the middle or right, and a call-to-action button (like "Sign Up" or "Get Started") on the far right. This works for most apps with 3-7 main pages.

Sidebar Navigation

Common in dashboards and admin panels. A vertical menu on the left side with icons and labels. Works well when you have many sections (10+) that users need to jump between frequently. This very learning platform uses sidebar navigation.

Bottom Tab Bar (Mobile)

The standard for mobile apps. 3-5 icons at the bottom of the screen. Think of how Instagram, Twitter, or any banking app works. If you're building a mobile-first experience, this is usually the way to go.

Hamburger Menu

The three horizontal lines that expand into a full menu when tapped. Useful for mobile views when you have too many links for a bottom tab bar. Just know that items hidden behind a hamburger menu get significantly fewer clicks than visible navigation items.

Pattern	Best For	Watch Out For
Top navbar	Marketing sites, blogs, landing pages	Gets crowded with too many links
Sidebar	Dashboards, admin panels, content platforms	Takes up horizontal space on small screens
Bottom tabs	Mobile-first apps, social apps	Limited to 3-5 items maximum
Hamburger menu	Mobile views, secondary navigation	Users click hidden items 50-70% less often

Try this prompt to get AI help with your navigation:

```
Here's my current app structure with these pages: [list your pages].
Suggest the best navigation pattern for both desktop and mobile.
Consider that my primary user action is [describe it]. Create the
React component for the navigation with Tailwind CSS styling.
```

Information Hierarchy: Guiding the Eye

When someone lands on your page, their eyes don't read every word from top to bottom. Research shows that people scan pages in an F-pattern or Z-pattern before deciding whether to read more deeply. This means you have about 3-5 seconds to communicate:

1. What is this? (headline)
2. Why should I care? (subheadline or value proposition)
3. What do I do next? (call-to-action)

Information hierarchy is the art of making the most important things visually prominent and the less important things visually subtle. You achieve this through:

- Size: Bigger text draws attention first
- Weight: Bold text stands out from regular text
- Colour: A bright button on a neutral background screams "click me"
- Spacing: More whitespace around an element makes it feel important
- Position: Top-left gets seen first (in left-to-right languages)

Here's a quick test: blur your eyes or squint at your app's homepage. The things that still stand out are your hierarchy. If the wrong things stand out (like a footer link instead of your main CTA), your hierarchy needs work.

The 3-Click Rule (And Why It Still Matters)

There's an old UX principle called the 3-click rule: a user should be able to reach any piece of content or complete any action within three clicks (or taps) from wherever they are.

Is it a hard rule? No. But the principle behind it is solid: every extra step you add between a user and their goal increases the chance they'll give up. Think of each click as a tollbooth on a highway. One or two tolls? Fine. Seven tolls? People take a different road.

Apply this to your app by mapping out user journeys:

- How many clicks from the homepage to signing up?
- How many clicks from the dashboard to the most-used feature?
- How many clicks to change account settings?
- How many clicks to get help or contact support?

If any of these journeys take more than 3-4 steps, look for ways to shorten them. Can you add a shortcut? Can you move a feature to a more prominent location? Can you combine two steps into one?

```
Analyze my app's user flow. A user lands on the homepage and wants to [describe the goal, e.g., \"purchase a subscription\"]. Map out every click and page transition required. If it takes more than 3 clicks, suggest ways to reduce the number of steps.
```

User Personas: Know Who You're Building For

A user persona is simply a fictional description of a typical user. You don't need a fancy template or a 20-page document. Even a few sentences can help you make better design decisions.

Here's a simple format:

Name: Sarah, 34, marketing manager

Tech comfort: Uses apps daily but has never written code

Goal: Wants to build a simple portfolio site for her freelance work

Frustrations: Gets overwhelmed by technical jargon, gives up when things look complicated

Device: Primarily uses her iPhone, occasionally her laptop

Now every time you make a design decision, you can ask: "Would Sarah understand this? Would Sarah find this useful? Would Sarah give up at this point?"

You don't need dozens of personas. Two or three is usually plenty. One for your primary user, one for a secondary user, and maybe one for an edge case.

```
I'm building [describe your app]. Help me create 3 simple user personas. For each persona, include: name, age, occupation, tech comfort level, primary goal, main frustration, and preferred device. Keep each persona to 3-4 sentences.
```

Accessibility: Building for Everyone

Accessibility (often shortened to a11y) means making your app usable by people with different abilities. This includes people who are visually impaired, colour-blind, have motor difficulties, or use screen readers.

But here's the thing: accessibility improvements almost always make the experience better for everyone. Higher colour contrast? Easier to read in sunlight. Larger tap targets? Easier for everyone on mobile. Clear labels? Helpful for non-native speakers too.

Here are the accessibility basics every builder should know:

Colour Contrast

Text needs enough contrast against its background to be readable. The Web Content Accessibility Guidelines (WCAG) recommend a minimum contrast ratio of 4.5:1 for normal text and 3:1 for large text. You can check this instantly with free tools like the WebAIM Contrast Checker.

Font Sizes

Body text should be at least 16px (1rem). Anything smaller becomes hard to read on mobile devices. Headings should be noticeably larger than body text to create that visual hierarchy we talked about.

Alt Text for Images

Every image that conveys meaning should have descriptive alt text. This helps screen readers describe the image to visually impaired users. Decorative images (like background patterns) can have empty alt text (alt="").

Keyboard Navigation

Some users navigate entirely with a keyboard (Tab, Enter, Escape). Make sure all interactive elements (buttons, links, form fields) are reachable and usable without a mouse.

Form Labels

Every input field needs a visible label. Placeholder text alone is not enough because it disappears once the user starts typing.

Accessibility Area	Quick Check	Tool to Help
Colour contrast	Can you read all text easily?	WebAIM Contrast Checker
Font size	Is body text at least 16px?	Browser DevTools
Alt text	Do meaningful images have descriptions?	Lighthouse audit
Keyboard nav	Can you Tab through the whole page?	Just try it yourself
Form labels	Does every input have a visible label?	Visual inspection

Audit my app for accessibility issues. Check for: colour contrast problems, missing alt text on images, form inputs without labels, elements that aren't keyboard-navigable, and font sizes below 16px. Provide specific fixes with code examples.

Dark Mode Considerations

Dark mode has gone from a nice-to-have to an expectation. Most modern apps offer it, and many users prefer it, especially at night or in low-light environments.

If you're using Tailwind CSS (which most projects in this course do), dark mode support is built in. You add dark: prefixes to your utility classes:

```
<div class="bg-white dark:bg-gray-900 text-gray-900 dark:text-gray-100">  
  <h1 class="text-black dark:text-white">Welcome</h1>  
</div>
```

But dark mode isn't just about inverting colours. Here are things to watch for:

- Pure black backgrounds (#000000) can feel harsh. Use dark grays (#111827, #1F2937) instead.
- Pure white text on dark backgrounds can cause eye strain. Slightly off-white (#F9FADF, #E5E7EB) is gentler.
- Images and logos might need light/dark variants. A dark logo on a dark background disappears.
- Shadows don't work well on dark backgrounds. Consider using borders or subtle glows instead.
- Colour choices need to work in both modes. That beautiful navy blue button might be invisible on a dark background.

Here's a great prompt to get AI to add dark mode to your existing components:

```
Add dark mode support to this component using Tailwind CSS dark:  
prefix classes. Make sure the colours work well in both light and  
dark modes. Use dark grays (not pure black) for backgrounds and  
slightly off-white for text. Check that all interactive elements  
remain clearly visible in dark mode.
```

Using AI to Audit Your UX

One of the most powerful things you can do with AI tools is ask them to critique your work. AI won't sugarcoat things or worry about hurting your feelings. It'll tell you straight if your navigation is confusing or your fonts are too small.

Here are several prompts you can use at different stages of your build:

Early stage (wireframe/planning):

Here's my planned page structure for a [type of app]. Review the navigation flow and information architecture. Are there any pages that should be combined? Any important pages missing? Is the user journey from landing to [primary action] as short as possible?

Mid-build (working prototype):

Review my app's current UI. I'll share screenshots of the main pages. For each page, evaluate: visual hierarchy, clarity of the call-to-action, readability, mobile-friendliness, and consistency with the other pages. Rate each area out of 10 and suggest specific improvements.

Pre-launch (final polish):

Pretend you're a first-time user who has never seen this app before. Walk through the experience of [describe a key user journey]. At each step, note anything that's confusing, unclear, or frustrating. Be brutally honest. I want to fix these issues before launch.

You can share your component files and ask:

Look at all the page components in /src/pages/. Evaluate the overall UX consistency across pages. Are headings, spacing, button styles, and layout patterns consistent? Create a list of inconsistencies and suggest a unified approach.

Practical Exercise: The 5-Minute UX Walk-Through

Here's something you can do right now that takes less than 5 minutes but can dramatically improve your app:

1. Open your app in a browser (or on your phone)
2. Pretend you've never seen it before. Clear your mind.
3. Start at the homepage and try to complete the most important action (sign up, find information, make a purchase)
4. Talk out loud as you go: "Okay, I see a big headline... I think I should click this button... now I'm on a page and I'm not sure what to do..."
5. Write down every moment of hesitation or confusion

Those hesitation points are your UX bugs. Fix those before you worry about adding new features, because no feature matters if people can't find it.

Key Takeaways

- You are not your user. You know your app inside and out. Your users are seeing it for the first time, often on a phone, often distracted. Design for them, not for yourself.
- Navigation is the skeleton of your app. Choose a pattern (top navbar, sidebar, bottom tabs) that matches your app type and keep it consistent across all pages.
- Visual hierarchy guides the eye. Use size, weight, colour, and spacing to make important elements stand out and less important elements recede.
- The 3-click rule is a useful guideline. Every extra step between a user and their goal increases the chance they'll abandon your app.
- Simple user personas keep you focused. Even two or three short persona descriptions help you make better design decisions.
- Accessibility benefits everyone. Colour contrast, font sizes, alt text, and keyboard navigation improvements make your app better for all users, not just those with disabilities.
- Dark mode is expected. Use Tailwind's dark: prefix, avoid pure black, and test that colours work in both modes.
- AI tools are excellent UX auditors. Ask them to critique your navigation, hierarchy, consistency, and accessibility. They'll be honest in ways your friends might not be.
- The 5-minute walk-through is your secret weapon. Pretend you've never seen your app, try to use it, and write down every moment of confusion. Fix those first.

Get Others to Give You Initial Feedback

Have you ever written an email, read it back five times, thought it was perfect, and then realised after sending it that you misspelled the recipient's name in the very first line? Your brain has a fascinating ability to fill in gaps and gloss over mistakes in things you've created yourself. It knows what you meant to write, so it reads what you meant instead of what's actually there.

The exact same thing happens when you build an app. You've been staring at it for days or weeks. You know every screen, every button, every flow. So when something is confusing, broken, or poorly placed, your brain just... skips right over it. You literally cannot see your own UX problems.

This is why getting feedback from other people isn't optional. It's not a nice-to-have. It's the single fastest way to find out what's actually wrong with your app before you launch it to a wider audience.

Why You Can't Judge Your Own UX

There's a well-documented psychological phenomenon called the curse of knowledge. Once you know something, you can't un-know it. You can't experience your app as a first-time user because you're not a first-time user. You already know:

- That the login button is behind the hamburger menu
- That you need to scroll down to see the pricing
- That the form requires a specific date format
- That the "Submit" button only works after you check that hidden checkbox

Your users don't know any of that. And they won't figure it out the way you think they will. They'll tap around for 10 seconds, get confused, and leave. Studies consistently show that users spend an average of 10-20 seconds on a new page before deciding whether to stay or go.

Here's another reason you need outside eyes: you're emotionally invested. You built this thing. You put hours into it. When someone says "I don't understand what this page is for," your first instinct is to defend it rather than fix it. That's completely natural, but it's also why fresh perspectives matter so much.

How to Recruit Testers

You don't need a formal user research lab. You don't need to hire anyone. You just need a handful of honest people who are willing to spend 10-15 minutes clicking around your app.

Here's a breakdown of where to find them:

Friends and Family (The Easy Win)

This is the lowest-friction option. Text three friends or family members a link to your app and ask them to try it out. The advantage: they'll do it because they like you. The disadvantage: they might be too

nice and not tell you what's really wrong.

To counter the "too nice" problem, give them specific tasks instead of asking "What do you think?" More on that below.

Online Communities

Reddit is goldmine for honest feedback. Subreddits like r/webdev, r/SideProject, r/startups, and r/design_critiques are full of people who enjoy reviewing other people's work. Post a link, describe what your app does, and ask for specific feedback. Be prepared: Reddit is not known for sugarcoating.

Discord communities are another excellent option. Many tech, design, and indie hacker Discord servers have channels specifically for sharing projects and getting feedback. The Cursor community Discord, various AI builder communities, and indie hacker groups are all good places to start.

Social Media

Twitter/X, LinkedIn, TikTok, and even Instagram can work if you have a following. Share a screenshot or short video of your app and ask for honest thoughts. You might be surprised how many people are willing to help.

Co-working Spaces and Meetups

If you work from a co-working space or attend local tech meetups, you have access to people who understand apps and websites but haven't seen yours. Ask someone during a coffee break: "Hey, can you spend 5 minutes trying my app and tell me if anything is confusing?"

Source	Pros	Cons	Best For
Friends & family	Easy to recruit, they'll actually do it	May be too polite	Quick first impressions
Reddit	Brutally honest, large audience	Can be harsh, no context about your goals	General UX critique
Discord communities	Engaged audience, ongoing relationship	Smaller reach	Technical feedback
Social media	Broad reach if you have followers	Low response rate	Quick visual feedback
Co-working / meetups	In-person observation, immediate follow-up	Limited availability	Detailed walkthroughs

What Questions to Ask Testers

The biggest mistake people make when asking for feedback is saying: "Hey, check out my app and let me know what you think!" This almost always results in vague responses like "Looks good!" or "Cool app!" which tells you absolutely nothing useful.

Instead, give testers specific tasks and specific questions. Here are examples:

Task-based questions (the most valuable kind):

- "Go to the homepage. Without clicking anything, tell me what this app does."
- "Try to create an account. Tell me if anything is confusing during the process."
- "Find the pricing page. How many clicks did it take?"

- "Try to change your profile picture. Were you able to do it without help?"
- "You want to cancel your subscription. Find out how to do that."

Experience questions (after they've used the app):

- "What was the most confusing part?"
- "Was there a moment where you didn't know what to do next?"
- "On a scale of 1-10, how easy was it to complete [specific task]?"
- "What would you change if you could change one thing?"
- "Would you use this app? Why or why not?"

Comparison questions (for context):

- "Have you used a similar app before? How does this compare?"
- "Does this remind you of any other website or app?"

The key is to watch what people do, not just listen to what they say. Someone might tell you "It was easy" while you watched them click the wrong button three times. Their actions reveal more than their words.

Creating a Simple Feedback Form

If you're sending your app to multiple testers, a simple feedback form helps you collect responses in one place and compare them. You don't need anything fancy. Google Forms is free and takes about 10 minutes to set up.

Here's a template you can adapt:

Section 1: First Impressions

- What do you think this app does? (Open text)
- Rate the homepage design from 1-5 (Scale)
- What was the first thing you noticed? (Open text)

Section 2: Task Completion

- Were you able to [task 1]? (Yes/No/Partially)
- Were you able to [task 2]? (Yes/No/Partially)
- Were you able to [task 3]? (Yes/No/Partially)
- Which task was hardest? Why? (Open text)

Section 3: Overall Experience

- What was the most confusing part? (Open text)

- What would you improve? (Open text)
- How likely are you to recommend this app to someone? 1-10 (Scale)
- Any other comments? (Open text)

You can ask AI to generate this form for you:

```
I'm building a [describe your app]. Create a Google Forms feedback questionnaire for early testers. Include first-impression questions, 3 specific task-completion questions relevant to my app, and overall experience questions. Keep it under 15 questions so testers don't get fatigued.
```

Tools for Collecting Feedback

Beyond simple forms, there are tools that can give you deeper insights into how people actually use your app:

Google Forms (Free)

Perfect for structured questionnaires. Easy to share via link. Responses automatically collected in a spreadsheet. This is where most people should start.

Hotjar (Free tier available)

Hotjar records actual user sessions on your website. You can watch recordings of people navigating your app, see heatmaps of where they click, and identify exactly where they get stuck. The free tier gives you enough to get started. This is incredibly eye-opening. You'll watch someone hover over the wrong button for 5 seconds before finding the right one, and you'll immediately know what needs to change.

Loom or Screen Recording

Ask testers to record their screen while using your app and narrate their thoughts out loud. Free Loom accounts allow short recordings. This gives you both the visual of what they're doing AND their real-time commentary. "Hmm, I'm not sure what this button does... let me try clicking it... oh, that wasn't what I expected." Gold.

Microsoft Clarity (Free)

Similar to Hotjar but completely free. Offers session recordings and heatmaps. Built by Microsoft, so it's reliable and well-maintained.

UserTesting.com (Paid)

If you want feedback from strangers who match your target audience, UserTesting connects you with real users. Expensive for indie projects, but worth knowing about for future reference.

Tool	Cost	What It Does	Best For
Google Forms	Free	Structured questionnaires	Collecting written feedback

Hotjar	Free tier (35 sessions/day)	Session recordings, heatmaps	Seeing where users get stuck
Loom	Free tier (25 videos)	Screen + voice recordings	Hearing tester's thought process
Microsoft Clarity	Completely free	Session recordings, heatmaps	Budget-friendly analytics
UserTesting	From \$49/session	Professional tester feedback	Targeted audience feedback

How to Handle Negative Feedback (Without Losing Your Mind)

Let's be honest: hearing criticism about something you've built is hard. You poured hours into this thing and someone just told you the navigation "makes no sense." Ouch.

But here's the reframe that changes everything: negative feedback is the most valuable kind. Positive feedback feels nice but doesn't help you improve. When someone says "I love it!" you learn nothing actionable. When someone says "I couldn't figure out how to sign up," you now know exactly what to fix.

Here's how to handle feedback constructively:

1. Write it all down before reacting

Don't respond to feedback in the moment. Collect it, write it down, and let it sit for a day. Your defensive instincts will fade and you'll see the feedback more clearly.

2. Look for patterns, not individual opinions

One person saying "the font is too small" might be personal preference. Five people saying it? That's a real problem. Focus on issues that multiple testers independently identify.

3. Separate the feedback from the person

Someone saying "this page is confusing" isn't saying you're a bad builder. They're giving you information about their experience. That information is a gift, even when it stings.

4. Prioritise by impact

Not all feedback is equally important. A confusing sign-up flow (which affects every new user) matters more than a slightly misaligned icon (which most people won't notice). Focus your energy where it has the biggest impact.

5. Thank your testers

Seriously. People who give you honest feedback are doing you a huge favour. Thank them, and if you can, show them the improvements you made based on their input. They'll be more likely to help again in the future.

Iterating Based on Feedback

Collecting feedback is only half the job. The other half is actually doing something with it. Here's a practical process for turning feedback into improvements:

Step 1: Compile all feedback in one place
Whether it's from forms, conversations, or recordings, get everything into a single document or spreadsheet.

Step 2: Categorise the issues
Group similar feedback together. Common categories include: navigation, visual design, content/copy, functionality, performance, and mobile experience.

Step 3: Rank by frequency and severity

Priority	Criteria	Example
Critical	Multiple testers, blocks core function	"I couldn't create an account"
High	Multiple testers, causes confusion	"I didn't know this was clickable"
Medium	Some testers, minor friction	"The spacing feels tight on mobile"
Low	One tester, cosmetic preference	"I'd prefer a blue button instead of green"

Step 4: Fix the critical and high items first
Don't try to fix everything at once. Start with the issues that affect the most users or block the most important actions.

Step 5: Test again
After making changes, get a fresh round of feedback. Did the fixes work? Did they create new problems? Iteration is a cycle, not a one-time event.

Here's a powerful way to use AI in this process:

```
Here's the feedback I received from 5 testers of my app:  
  
[Paste all feedback here]  
  
Help me prioritise these issues. Group similar feedback together,  
rank each group by severity and frequency, and suggest the order  
I should fix them in. For the top 3 issues, suggest specific  
code changes or design improvements.
```

Using AI to Analyze and Act on Feedback

AI tools are exceptionally good at processing feedback and turning it into action items. Here are some prompts that can save you hours:

Synthesising large amounts of feedback:

```
I collected feedback from 10 users testing my [type of app].  
Here are their responses: [paste feedback]. Identify the top 5  
recurring themes and suggest a prioritised fix for each one.
```

Turning feedback into code changes:

```
Testers said my navigation is confusing. Here's my current  
navbar component: [paste code]. Based on standard UX patterns,  
rewrite this component to be clearer and more intuitive. Add  
visual indicators for the active page and improve mobile layout.
```

Creating a feedback loop template:

```
Create a simple feedback tracking system I can use in a  
Markdown file. Include columns for: issue description, who  
reported it, severity, status (open/in-progress/fixed), and  
date fixed. Pre-populate it with common categories.
```

Generating follow-up questions:

```
A tester said: "The checkout process was confusing." That's  
vague and I need more detail. Write 5 follow-up questions I  
can ask this tester to understand exactly what was confusing  
and at which step.
```

In Cursor or Claude Code, you can go even further:

```
I'm going to share feedback from my beta testers along with  
the relevant component files. For each piece of feedback,  
identify which file needs to change and make the specific  
code modifications to address the issue. Explain each change  
you make.
```

When to Seek Feedback (Timing Matters)

Don't wait until your app is "finished" to get feedback. The earlier you get it, the cheaper and easier it is to make changes. Here's a rough timeline:

Before you build (idea stage):

Describe your app idea to 3-5 people. Do they understand it? Are they excited? Would they use it? If most people look confused when you explain your idea, refine your concept before writing any code.

After your first few pages are built:

Share the basic flow with a couple of friends. Can they navigate from page A to page B? Does the layout make sense? This catches structural problems early.

Before launch (feature-complete):

This is the big feedback round. Get 5-10 people to thoroughly test all major features. Give them specific tasks. Collect structured feedback. Fix the critical issues.

After launch (ongoing):

Add a simple feedback mechanism to your app. A "Report a bug" link in the footer, a feedback button, or even just a contact email. Your users will find problems you never thought of.

Practical Exercise: Your First Feedback Round

Do this today:

1. Pick 3 people who haven't seen your app (friends, family, colleagues)
2. Send them the link with this message: "I'm building an app and would love your honest feedback. Can you spend 5-10 minutes trying it? Here are 3 things I'd like you to try: [list 3 specific tasks]. After, tell me: what was confusing? What would you change?"
3. Collect their responses in a Google Doc or spreadsheet
4. Identify the top 3 issues that came up most frequently
5. Fix them using AI to help with the code changes

That's it. Five steps. You'll learn more about your app's UX in one feedback round than in a week of staring at it yourself.

Key Takeaways

- You can't judge your own UX. The curse of knowledge means you literally cannot see problems that are obvious to new users. Outside feedback is essential, not optional.
- You don't need fancy tools to get feedback. Three friends with a link and some specific tasks will reveal 80% of your major UX issues.
- Ask specific questions, not vague ones. "Try to create an account and tell me what's confusing" beats "What do you think?" every single time.
- Look for patterns, not individual opinions. One person disliking your colour scheme is preference. Five people unable to find the sign-up button is a problem.
- Negative feedback is the most valuable kind. It stings, but it's the only feedback that tells you exactly what to fix.
- Use tools strategically. Google Forms for structured feedback, Hotjar or Clarity for session recordings, Loom for narrated walkthroughs.
- Iterate in cycles. Collect feedback, prioritise, fix, then test again. UX improvement is ongoing, not a one-time event.
- AI is excellent at processing feedback. Feed it raw tester feedback and ask it to prioritise issues, suggest fixes, and generate code changes.
- Start early and keep going. Don't wait until launch. Get feedback at every stage, from idea to post-launch.

Make It Responsive with the Latest Standards

Picture two restaurants. The first has one enormous banquet table that seats 20 people. Whether you're a couple on a date night or a family of four, you're sitting at that same giant table with acres of empty space around you. The second restaurant has tables of all sizes: intimate two-seaters by the window, four-tops in the middle, and larger tables in the back for groups. Every guest gets a table that fits their party.

Responsive design is the second restaurant. Your app should look and feel right whether someone is viewing it on a 27-inch desktop monitor, a 13-inch laptop, a 10-inch tablet, or a 6-inch phone. The content is the same, but the layout adapts to fit the screen. No giant empty spaces. No tiny text. No horizontal scrolling. Just an experience that feels like it was designed specifically for whatever device the user happens to be holding.

And here's the reality check: depending on your app, 70-80% of your users might be on mobile. If your app doesn't work well on a phone, you're ignoring the majority of your audience.

In this module, we really cover a lot of topics here. A friendly reminder as always, if you see recommendations below, whether you understand them or not, give them to AI! Give it some context to work with and go from there. You can't be all things when it comes to apps, UI, UX, and design. So lean on AI where you can!

Mobile-First Design Philosophy

There are two ways to approach responsive design:

1. Desktop-first: Design for big screens, then figure out how to squeeze everything onto smaller screens
2. Mobile-first: Design for phones first, then expand and enhance for larger screens

Mobile-first is the modern standard, and for good reason. When you start with a small screen, you're forced to prioritise. You can't fit everything above the fold on a 375px-wide screen, so you have to decide: what's the most important content? What can be tucked away? What's the primary action?

This constraint actually produces better designs. When you start with desktop, you tend to fill all that space with stuff, and then cramming it onto mobile becomes a nightmare. When you start with mobile, you build a focused, clean experience that then gracefully expands on bigger screens.

In practical terms, mobile-first means your base CSS styles are written for mobile screens, and you add complexity as the screen gets larger. With Tailwind CSS, this is the default behaviour, as we'll see shortly.

CSS Media Queries: The Foundation

Before we get into Tailwind's responsive utilities (which you'll use most of the time), it helps to understand what's happening under the hood. Responsive design is powered by CSS media queries, which are rules that say "apply these styles only when the screen meets certain conditions."

Here's what a basic media query looks like in plain CSS:

```
* Base styles (mobile) *
.container {
  display: flex;
  flex-direction: column;
  padding: 16px;
}

* When screen is 768px or wider (tablet and up) *
@media (min-width: 768px) {
  .container {
    flex-direction: row;
    padding: 32px;
  }
}
```

This says: "By default, stack things vertically with small padding. But if the screen is at least 768px wide, switch to horizontal layout with more padding."

The min-width approach is mobile-first: you write styles for the smallest screen first, then add modifications as the screen grows. If you see max-width media queries instead, that's desktop-first thinking (starting big and making things smaller).

You don't need to write raw media queries when using Tailwind, but understanding them helps you debug when things don't look right and helps you communicate with AI tools about what you want.

Tailwind Responsive Utilities: Your Daily Driver

If you're using Tailwind CSS (and most projects in this course do), responsive design becomes remarkably straightforward. Tailwind uses breakpoint prefixes that correspond to common screen sizes:

Prefix	Minimum Width	Typical Device
(none)	0px	Mobile phones
sm:	640px	Large phones, small tablets
md:	768px	Tablets
lg:	1024px	Laptops, small desktops
xl:	1280px	Desktops

The key insight: unprefixed classes apply to all screen sizes, and prefixed classes apply at that breakpoint and above. This is mobile-first by default.

Here's a practical example:

```
<div class="flex flex-col md:flex-row gap-4 p-4 md:p-8">
  <div class="w-full md:w-1/2">Left content</div>
  <div class="w-full md:w-1/2">Right content</div>
</div>
```

What this does:

- On mobile (default): Stacks the two divs vertically (flex-col), full width each (w-full), with 16px padding (p-4)
- On tablets and up (md: prefix): Places them side by side (md:flex-row), each taking half the width (md:w-1/2), with 32px padding (md:p-8)

Here's another common pattern for text sizing:

```
<h1 class="text-2xl md:text-4xl lg:text-5xl font-bold">
  Welcome to My App
</h1>
<p class="text-sm md:text-base lg:text-lg">
  A short description of what this app does.
</p>
```

The heading starts at a readable mobile size and grows larger on bigger screens. The paragraph text does the same. No media queries to write. No separate stylesheets to maintain.

Flexbox and Grid: The Layout Engines

Two CSS layout systems power most responsive designs: Flexbox and CSS Grid. You don't need to be an expert in either, but understanding when to use which one will save you (and your AI assistant) a lot of back-and-forth.

Flexbox is best for one-dimensional layouts: a row of buttons, a navigation bar, a card that stacks its content vertically. It's great when you want items to flow in one direction and wrap or adjust their size based on available space.

CSS Grid is best for two-dimensional layouts: a grid of cards, a page layout with header, sidebar, and main content, or any design where you need precise control over both rows and columns.

Feature	Flexbox	CSS Grid
Direction	One direction (row or column)	Two directions (rows AND columns)
Best for	Navigation bars, card content, button groups	Page layouts, card grids, dashboards
Tailwind classes	flex, flex-row, flex-col, gap, justify, items	grid, grid-cols-*, grid-rows-*, gap, col-span-*
Responsive?	Very. Items naturally wrap with flex-wrap	Very. Change column count at breakpoints

A common responsive grid pattern:

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  <div class="bg-white p-6 rounded-lg shadow">Card 1</div>
  <div class="bg-white p-6 rounded-lg shadow">Card 2</div>
  <div class="bg-white p-6 rounded-lg shadow">Card 3</div>
  <div class="bg-white p-6 rounded-lg shadow">Card 4</div>
  <div class="bg-white p-6 rounded-lg shadow">Card 5</div>
  <div class="bg-white p-6 rounded-lg shadow">Card 6</div>
</div>
```

This creates:

- Mobile: 1 card per row (stacked vertically)
- Tablet: 2 cards per row
- Desktop: 3 cards per row

All with consistent spacing between them. This is one of the most commonly used patterns in modern web development.

Testing on Different Screen Sizes

You don't need a drawer full of devices to test responsive design. Your browser's built-in developer tools have everything you need.

Chrome DevTools (the most popular method):

1. Right-click anywhere on your page and select "Inspect" (or press F12)
2. Click the device toggle icon (looks like a phone and tablet overlapping) in the top-left of DevTools
3. Now you can resize your viewport to any size, or pick from preset devices like iPhone 14, iPad, Galaxy S21, etc.
4. Your page re-renders in real-time as you resize

Firefox Responsive Design Mode:

Press Ctrl+Shift+M (Cmd+Shift+M on Mac) to toggle responsive design mode. Similar to Chrome but with a slightly different interface.

Real device testing:

When you're running your dev server, you can usually access it from your phone if both devices are

on the same WiFi network. Run `npm run dev -- --host` and your terminal will show a network URL (like `http://192.168.1.x:5173`). Open that URL on your phone's browser.

Key screen sizes to test:

Device	Width	What to Check
iPhone SE	375px	Smallest common phone
iPhone 14+	390px	Most popular phone size
iPad	768px	Tablet breakpoint
iPad Pro	1024px	Large tablet / small laptop
Laptop	1280px-1440px	Standard laptop screens
Desktop	1920px	Full HD monitors

Don't just check if things fit. Check if they're usable. Can you tap buttons easily? Is text readable without zooming? Does the navigation work? Are images loading at appropriate sizes?

Common Responsive Patterns

Certain responsive patterns appear in nearly every modern app. Here are the ones you'll implement most frequently:

Hamburger Menu

On desktop, your navigation shows all links in a horizontal bar. On mobile, those links collapse into a hamburger icon (three stacked lines) that expands into a full-screen or slide-out menu when tapped.

```
Create a responsive navigation bar for my React app using Tailwind.
On desktop (md: and up), show all navigation links horizontally.
On mobile, collapse them into a hamburger menu that slides in
from the right when tapped. Include smooth animation for the
menu opening and closing. Use React state to toggle the menu.
```

Stacked-to-Side-by-Side Layout

Content that sits side by side on desktop stacks vertically on mobile. This is the flex-col md:flex-row pattern we saw earlier. Used for feature sections, testimonial blocks, pricing comparisons, and about pages.

Responsive Images

Images should scale with their container and not overflow. The Tailwind classes `w-full h-auto` handle this for most cases. For hero images or backgrounds, you might use `object-cover` to maintain aspect ratio without stretching.

Collapsible Sidebars

Dashboard apps often have a sidebar that's always visible on desktop but collapses into an overlay or slides away on mobile. This keeps the main content area usable on small screens.

Responsive Tables

Tables are notoriously tricky on mobile. Common solutions include: horizontal scrolling (overflow-x-auto), converting table rows into stacked cards on mobile, or hiding less important columns.

```
I have a data table component showing user information with 6
columns. On desktop it should display as a normal table. On
mobile (below md: breakpoint), convert it to a card layout
where each row becomes a card with the data stacked vertically.
Use Tailwind CSS classes.
```

The Viewport Meta Tag (Don't Forget This)

There's one tiny line of HTML that makes all of responsive design work, and it's already in your project if you used a modern framework like Vite or Next.js:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This tells the browser: "Render this page at the actual width of the device, not at some zoomed-out desktop width." Without this tag, mobile browsers will try to render your page as if it were a 980px desktop site and zoom out to fit, making everything tiny.

If you're using Vite with React, this tag is already in your index.html. If you're using Next.js, it's handled automatically. But if you ever start a project from scratch and wonder why your responsive styles aren't working on a real phone, check this tag first.

Touch Targets: The 44px Rule

Apple's Human Interface Guidelines recommend that all tappable elements be at least 44x44 pixels. Google's Material Design guidelines are similar. This isn't arbitrary: it's the minimum size for a human fingertip to tap accurately.

Common violations of this rule:

- Small icon buttons without enough padding
- Links that are just underlined text with no extra tap area
- Close buttons (X) on modals that are 16x16 pixels
- Form checkboxes that rely on the default browser size

With Tailwind, you can ensure proper touch targets with padding:

```
<!-- Too small for mobile -->
<button class="p-1 text-sm">Submit</button>

<!-- Properly sized for touch -->
<button class="p-3 text-base min-h-[44px] min-w-[44px]">Submit</button>
```

For icon buttons specifically:

```
<button class="p-3 rounded-full hover:bg-gray-100" aria-label="Close">
  <X class="w-5 h-5">
</button>
```

The icon itself might be 20x20 pixels, but the 12px padding on each side brings the total tappable area to 44x44 pixels.

Performance on Mobile

Mobile users often have slower connections and less powerful devices than desktop users. Performance matters even more on mobile. Here are the key optimisations:

Image Optimisation

Images are usually the heaviest assets on a page. For responsive design:

- Use modern formats like WebP (supported by all modern browsers)
- Serve different image sizes for different screens. A hero image doesn't need to be 3000px wide on a phone.
- Add `loading="lazy"` to images below the fold so they only load when the user scrolls to them

Lazy Loading Components

In React, you can lazy-load components that aren't immediately visible:

```
const HeavyChart = React.lazy(() => import('./HeavyChart'));
```

This means the chart component's code only downloads when it's actually needed, not on initial page load.

Minimising JavaScript

Mobile devices parse JavaScript slower than desktops. Keep your bundles small by:

- Only importing what you need from libraries
- Using tree-shaking (Vite does this automatically)

- Avoiding massive dependencies when a lighter alternative exists

```
Audit my project's bundle size. Identify the largest dependencies
and suggest lighter alternatives where possible. Also check for
any imports that could be more specific (e.g., importing all of
lodash vs just the functions I need).
```

Using AI to Make Components Responsive

One of the most practical uses of AI in your workflow is converting desktop-only components into responsive ones. Here are prompts that work well across Cursor, Claude Code, and other tools:

Converting an existing component:

```
This component currently only looks good on desktop. Make it
fully responsive using Tailwind CSS breakpoints. On mobile,
the content should stack vertically with appropriate spacing.
On tablet (md:), use a two-column layout. On desktop (lg:),
use the current layout. Ensure all touch targets are at least
44px and text is readable without zooming.
```

Building a responsive layout from scratch:

```
Create a responsive pricing page with 3 plan cards. On mobile,
the cards should stack vertically. On tablet, show 2 cards per
row with the third below. On desktop, show all 3 in a row. The
middle card should be visually highlighted as the recommended
plan. Use Tailwind CSS and make sure it works in dark mode too.
```

Fixing responsive issues:

```
On mobile, my page has these problems: [describe issues, e.g.,
"text overflows the container", "button is too small to tap",
"image is cut off on the right"]. Here's the component code.
Fix these responsive issues while keeping the desktop layout
unchanged.
```

Full page audit:

```
Review all components in my /src/components/ directory. For each
one, check if it uses responsive Tailwind classes (sm:, md:, lg:).
List any components that are NOT responsive and suggest the
specific classes needed to make them work on mobile.
```

A Quick Responsive Design Checklist

Before you consider any page "done," run through this checklist to ask AI:

- Does the page look good at 375px wide (smallest common phone)?
 - Does text wrap properly without horizontal scrolling?
 - Are all buttons and links at least 44x44px on touch screens?
 - Does the navigation work on mobile (hamburger menu, bottom tabs, etc.)?
 - Are images scaling properly and not overflowing their containers?
 - Is body text at least 16px on all screen sizes?
 - Do forms work on mobile (inputs are full-width, keyboard doesn't cover fields)?
 - Does the layout gracefully transition between breakpoints (no awkward in-between states)?
 - Are there no horizontal scroll bars at any screen size?
 - Does the page load quickly on a throttled connection (check with Chrome DevTools Network tab)?
-

Key Takeaways

- Mobile-first is the modern standard. Design for phones first, then enhance for larger screens. This forces you to prioritise content and produces cleaner designs.
- Tailwind's responsive prefixes make it easy. Unprefixed classes apply to all sizes; sm:, md:, lg:, xl:, and 2xl: apply at that breakpoint and above.
- Flexbox for one dimension, Grid for two. Use Flexbox for navigation bars and card content. Use Grid for page layouts and card grids.
- Test with browser DevTools. Chrome's device toggle lets you preview any screen size instantly. Test at 375px, 768px, and 1280px at minimum.
- The hamburger menu is your mobile navigation friend. Collapse desktop navigation into a hamburger on mobile. But remember: hidden items get fewer clicks.
- Touch targets must be at least 44x44 pixels. Pad your buttons, links, and interactive elements generously for mobile users.
- Optimise images for mobile. Use WebP format, serve appropriate sizes with srcset, and add lazy loading for below-the-fold images.
- Don't forget the viewport meta tag. Without it, mobile browsers will zoom out your page to fit a desktop-width viewport.
- AI tools are excellent at making components responsive. Share your component code and describe how you want it to behave at different screen sizes. The AI will add the right breakpoint classes.

Try Your Hand at Figma

Imagine you're renovating your kitchen. You wouldn't just start ripping out cabinets and hoping for the best. You'd sketch out what you want first, maybe on a napkin, maybe with a proper floor plan. Where does the fridge go? Is there enough counter space? Will the oven door block the walkway? You'd figure all of this out on paper before spending money on materials and labour.

Figma is that sketch for your app. It's a free design tool that lets you lay out your pages, move things around, try different arrangements, and get a visual sense of how your app will look and feel, all before you write a single line of code. You don't need to be a designer to use it. You don't need artistic talent. You just need to be willing to drag some rectangles around on a screen.

And here's why that matters for you as someone building with AI tools: the better you can visualise what you want, the better your prompts will be, and the better your AI-generated code will turn out.

What Figma Is (And Why Non-Designers Should Care)

Figma is a browser-based design tool. You don't need to install anything. You go to figma.com, create a free account, and you're designing in your browser. It's used by professional designers at companies like Google, Microsoft, and Airbnb, but it's also perfectly suited for solo builders and non-designers who just need to plan out their interfaces.

Why should you care about Figma if you're not a designer?

1. It saves you time with AI tools

When you have a Figma mockup, you can describe your design precisely to Cursor, Claude Code, or Lovable. Instead of saying "make me a nice landing page," you can say "build this exact layout I designed in Figma, with the hero section taking up the full viewport, three feature cards below it, and a footer with four columns." The more specific your description, the closer the AI gets on the first try.

2. It catches UX problems before they exist in code

Rearranging rectangles in Figma takes seconds. Rearranging React components takes much longer. If you discover that your sidebar is too wide or your form has too many fields, it's infinitely easier to fix in Figma than in code.

3. It helps you communicate with others

If you're building for a client, a friend, or a team, a visual mockup communicates more in 5 seconds than a paragraph of text. "Here's what the dashboard will look like" is much more effective when accompanied by an actual picture of the dashboard.

4. It's free

Figma's free tier is genuinely generous. You get 3 Figma files with unlimited pages in each. For a solo builder working on one or two projects, that's more than enough.

Figma's Free Tier: What You Get

Feature	Free Tier	Professional (Paid)
Figma files	3 files	Unlimited
Pages per file	Unlimited	Unlimited
Editors	You + 1 collaborator	Unlimited
Version history	30 days	Unlimited
Components & styles	Basic	Advanced libraries
Plugins	Full access	Full access
Prototyping	Full access	Full access
Community resources	Full access	Full access

For learning and building personal projects, the free tier is all you need. You'd only need to upgrade if you're working with a larger team or managing many projects simultaneously.

Basic Figma Concepts (The 15-Minute Crash Course)

You don't need to master Figma. You need to understand four concepts, and you'll be productive enough to plan any app.

Frames

A frame is like a container or a screen. When you start designing a page, you create a frame that represents the screen size. Figma has presets: "iPhone 14 Pro" (393x852), "Desktop" (1440x900), "iPad" (810x1080), etc. Think of each frame as one page or screen of your app.

To create a frame: Press F on your keyboard, or select the Frame tool from the toolbar. Then either click and drag to draw a custom size, or pick a preset from the right panel.

Shapes and Text

The basic building blocks. Press R for a rectangle, T for text, O for an ellipse (circle). Most wireframes are just rectangles with text inside them. A button? Rectangle with text. A card? Rectangle with text and maybe an image placeholder. A navigation bar? Long rectangle across the top with text links inside.

Auto Layout

This is Figma's version of Flexbox. When you select multiple elements and add Auto Layout (Shift+A), Figma automatically spaces them evenly and adjusts when you add or remove items. It's like `display: flex; gap: 16px;` but visual.

Auto Layout is the single most useful Figma feature for non-designers. It keeps your layouts consistent without you having to manually position every element.

Components

When you have an element you'll reuse (like a button, a card, or a navigation bar), you can turn it into a component. Then when you place copies of that component across your design, changing the original (called the "main component") updates all the copies. This is exactly like React components, and that's not a coincidence.

To create a component: Select the element and press Ctrl+Alt+K (Cmd+Option+K on Mac).

Creating a Simple Wireframe (Step by Step)

Let's walk through creating a basic wireframe for a landing page. This should take about 20 minutes.

Step 1: Create a frame

Press F, then select "Desktop" (1440x900) from the right panel. This is your canvas.

Step 2: Add a navigation bar

Draw a rectangle across the top (full width, about 64px tall). Add text for your logo on the left and navigation links on the right ("Home", "Features", "Pricing", "Contact"). Add a rectangle for a CTA button ("Get Started").

Step 3: Add a hero section

Below the nav, draw a large rectangle (full width, about 500px tall). Add a large text element for your headline, a smaller one for a subheadline, and a button rectangle for the primary CTA.

Step 4: Add feature cards

Below the hero, create three rectangles side by side (each about 400px wide). Inside each, add a small circle (icon placeholder), a heading, and a short description. Select all three and add Auto Layout to space them evenly.

Step 5: Add a footer

At the bottom, draw a rectangle (full width, about 200px). Add text for copyright, links, and social media placeholders.

That's it. You now have a wireframe. It won't win any design awards, but it gives you a clear visual plan for what you're going to build. And when you show it to your AI tool, the results will be dramatically better than "make me a landing page."

Using Figma Templates and Community Resources

Here's a secret that professional designers use constantly: you don't start from scratch. Figma has a massive community of designers who share free templates, UI kits, wireframe kits, and design systems.

To access community resources:

1. Go to figma.com/community
2. Search for what you need ("landing page wireframe", "dashboard UI kit", "mobile app template")
3. Click "Open in Figma" to duplicate the file into your account
4. Customise it to match your project

Some popular free resources worth bookmarking:

Resource	What It Is	Best For
Wireframe kits	Pre-built wireframe components (buttons, forms, cards, navbars)	Quick wireframing without drawing from scratch
UI kits	Polished, ready-to-use design components	When you want your mockup to look close to the final product
Landing page templates	Complete landing page designs	Starting point for marketing sites
Dashboard templates	Admin panel and dashboard layouts	Planning complex data-heavy interfaces
Icon sets	Collections of consistent icons	Adding visual elements to your wireframes

Using a template doesn't mean your app will look generic. Think of it as scaffolding: you start with the structure and then customise the content, colours, and layout to make it yours.

Prototyping Basics: Making Your Design Interactive

Figma isn't just for static images. You can connect your frames together to create a clickable prototype, which is like a slideshow that responds to clicks. This lets you "use" your app before it exists.

Here's how prototyping works:

1. Switch to the Prototype tab in the right panel
2. Select an element you want to be clickable (like a button)
3. Drag the blue circle that appears on the right side of the element to the frame (page) it should link to
4. Set the interaction: "On Click" is the most common trigger
5. Set the animation: "Navigate to" with "Smart Animate" for smooth transitions

Now when you hit the Play button (top-right corner), you can click through your design as if it were a real app. Click the "Get Started" button on your landing page and it takes you to the sign-up page. Click "Dashboard" in the nav and it shows the dashboard.

This is incredibly useful for:

- Testing your user flow before building anything
- Showing stakeholders or clients what the final product will feel like
- Identifying missing pages ("Wait, what happens after they click Submit?")
- Getting feedback (share the prototype link and let people click through it)

Figma to Code: Bridging the Gap

Once you have a Figma design, how do you turn it into real code? There are several approaches:

Manual translation (most common for learners)

You look at your Figma design and describe it to your AI tool. This is the approach most people in this course will use, and it works extremely well.

```
Here's a description of my Figma wireframe for the landing page:
```

- Navigation bar: logo on the left, 4 text links centered, "Get Started" button on the right with primary colour
- Hero section: full-width, centered headline (48px), subheadline below (18px), two buttons side by side ("Start Free Trial" primary, "Watch Demo" secondary)
- Features section: 3 cards in a row, each with an icon, heading, and 2-line description
- Footer: 4 columns with links, copyright at the bottom

```
Build this as a React component using Tailwind CSS. Make it responsive (stack everything vertically on mobile).
```

Figma's "Dev Mode" (Inspect panel)

Even on the free tier, you can select any element in Figma and see its properties: dimensions, colours, fonts, spacing. This helps you communicate exact values to your AI tool. "The card has 24px padding, a 1px border with colour #E5E7EB, and 8px border radius" is much more precise than "make it look like a card."

Figma plugins that generate code

Plugins like "Figma to Code (HTML, Tailwind CSS)" or "Locofy" can automatically export Figma designs as HTML/CSS code. The output usually needs cleanup, but it can be a useful starting point. You can paste the generated code into Cursor and ask it to refactor into proper React components.

Screenshot approach

The simplest method: take a screenshot of your Figma design and drag it into Cursor's chat or Claude Code's conversation. AI tools are increasingly good at interpreting visual designs. "Build this exact layout from the screenshot" combined with an image often produces remarkably accurate results.

AI Plugins for Figma

Figma has a plugin ecosystem, and several AI-powered plugins can speed up your design process:

Magician - Generates icons, copy, and images from text prompts directly in Figma. Need a placeholder image of a dashboard? Just describe it.

Ando - An AI design assistant that helps with layout suggestions, colour palettes, and component creation inside Figma.

Content Reel - Fills your designs with realistic placeholder content (names, avatars, addresses) so your wireframes look more like real apps.

Figma AI (built-in) - Figma itself has been adding AI features for generating designs from text descriptions, renaming layers automatically, and suggesting layout improvements.

To install a plugin: Click the Resources icon (the small diamond shape) in the toolbar, search for the plugin, and click "Run." Plugins work directly inside your Figma file.

Describing Figma Designs to AI Coding Tools

The bridge between Figma and code is your description. So, get those screenshots ready, and complement them with a detailed description. A screenshot doesn't move remember so you'll need to describe the elements in motion as well. The better you describe your design, the better the AI output. Here's a framework for describing any Figma page to an AI tool:

1. Start with the overall structure

"This page has a navbar at the top, a hero section, a three-column features section, a testimonials carousel, and a footer."

2. Describe each section with specifics

"The hero section is full-width with a gradient background from blue-600 to purple-600. The headline is white, 48px, centered. Below it is a 20px subheadline in white with 60% opacity. Two buttons below: primary (white background, dark text) and secondary (transparent with white border)."

3. Include responsive behaviour

"On mobile, the feature cards should stack vertically instead of sitting in a row. The hero headline should drop to 32px."

4. Mention interactions

"The navbar should become sticky when you scroll past the hero. The hamburger menu should slide in from the right on mobile."

5. Reference specific values when you have them

"Card padding is 24px. Gap between cards is 32px. Border radius is 12px. Shadow is md (Tailwind's shadow-md)."

Here's a complete prompt template:

I've designed a page in Figma. Here's the layout:

[Describe overall structure]

Specific details:

- [Section 1 details including colours, sizes, spacing]
- [Section 2 details]
- [Section 3 details]

Responsive behaviour:

- [How it should change on mobile]
- [How it should change on tablet]

Build this as a React component with Tailwind CSS. Include dark mode support. Make all interactive elements accessible.

When Figma Is Overkill vs Essential

Figma is a powerful tool, but you don't always need it. Here's a quick decision framework:

Skip Figma when:

- You're building a simple personal project and have a clear mental picture
- You're following a tutorial or template closely
- The project is a quick prototype you'll throw away
- You're building something with a well-established layout (like a standard blog)

Use Figma when:

- You're building for a client who needs to approve designs before development
- You're working with a team and need to communicate visual decisions
- The layout is complex (dashboards, multi-step forms, data-heavy pages)
- You're unsure about the layout and want to experiment visually before coding
- You want to test your user flow with a clickable prototype before building
- You're planning a significant redesign of an existing app

For most learners in this course, a simple Figma wireframe for your main pages (homepage, dashboard, key feature page) is the sweet spot. You don't need pixel-perfect mockups. Rough wireframes that show layout, hierarchy, and navigation flow are enough to significantly improve your AI-assisted building process.

Practical Exercise: Your First Figma Wireframe

Spend 30 minutes on this exercise:

1. Go to figma.com and create a free account (if you haven't already)
2. Create a new file and name it after your project
3. Create a Desktop frame (1440x900) and a Mobile frame (393x852) side by side
4. Wireframe your homepage on the Desktop frame using only rectangles and text. Include: navigation, hero section, one content section, and a footer.
5. Wireframe the same page on the Mobile frame, adjusting the layout to stack vertically
6. Share the file with someone and ask: "Does this layout make sense for a [type of app]?"

Then use your wireframe to write a detailed prompt for Cursor, Claude Code, or Lovable:

```
Based on my Figma wireframe, here's the layout for my homepage:  
[Describe each section based on your wireframe]
```

```
Build this as a responsive React page using Tailwind CSS.  
The desktop layout should match my wireframe. On mobile,  
everything should stack vertically with appropriate spacing.
```

You'll be amazed at how much better the AI output is compared to a vague "build me a homepage" prompt.

Key Takeaways

- Figma is a free, browser-based design tool that lets you plan your app visually before writing code. No design experience required.
- You only need four concepts to be productive: frames (screens), shapes and text (building blocks), Auto Layout (automatic spacing), and components (reusable elements).
- Wireframes don't need to be pretty. Rough layouts with rectangles and text are enough to dramatically improve your AI prompts and catch UX problems early.
- The Figma community is a goldmine. Free templates, UI kits, and wireframe kits let you start with professional-quality structures and customise them.
- Prototyping lets you test your user flow by creating clickable connections between pages. Find missing pages and confusing flows before building.
- Describing your Figma design precisely is the bridge to great AI-generated code. Include structure, specific values (sizes, colours, spacing), and responsive behaviour.
- Screenshots work too. Dragging a Figma screenshot into Cursor or Claude Code and saying "build this" is a valid and effective workflow.
- Use Figma for complex layouts and client work. Skip it for quick personal projects where you have a clear mental picture.
- 30 minutes in Figma can save you hours in code. Rearranging rectangles is infinitely faster than restructuring React components.

Come Up with Scenarios to Test Features

Imagine you're a car manufacturer and you've just built a brand-new vehicle. It looks great in the showroom. The paint is gleaming, the seats are comfortable, the dashboard lights up beautifully. But would you sell that car to a customer without driving it first? Without checking that the brakes work in the rain? Without seeing what happens when someone slams the door too hard or fills the boot past capacity? Of course not. You'd test every single thing before a customer ever sits in that driver's seat.

Your app is no different. Before anyone else uses it, you need to think about all the ways people will interact with it, including the ways you didn't intend. What happens when someone types their email address wrong? What happens when the internet drops halfway through a form submission? What happens when someone resizes their browser to a tiny window?

Testing isn't about finding proof that your app works. It's about finding proof that it doesn't break. And the secret to good testing is coming up with the right scenarios before you start clicking around randomly.

Why Feature Testing Matters Before Launch

Every app has bugs. That's not a controversial statement. Even apps built by massive teams at Google, Apple, and Microsoft ship with bugs. The goal isn't zero bugs; it's catching the ones that matter most before your users find them.

Here's what happens when you skip testing:

- A user signs up but never receives their confirmation email (you lose them forever)
- Someone fills out a 10-field form, hits submit, and gets an error with no explanation (they won't try again)
- Your checkout flow works perfectly on Chrome but breaks on Safari (you lose every iPhone user)
- A user with a slow 3G connection watches your page load for 15 seconds and gives up
- Someone types `<script>alert('hacked')</script>` into your search bar and... well, that's a whole different problem

Each of these scenarios is preventable if you think about them in advance. The time you spend testing before launch saves you ten times that amount in panicked bug-fixing, lost users, and reputation damage after launch.

Writing Test Scenarios in Plain English

You don't need to write code to test your app. You don't need a testing framework or a QA department. You need a list of scenarios written in plain English that describe what a user might do and what should happen.

A test scenario has three parts:

1. Given (the starting state): Where is the user? What have they done so far?
2. When (the action): What does the user do?
3. Then (the expected result): What should happen?

Here are some examples:

Scenario: Successful sign-up

- Given I'm on the registration page
- When I enter a valid email, a strong password, and click "Create Account"
- Then I should see a success message and receive a confirmation email

Scenario: Sign-up with an existing email

- Given I'm on the registration page
- When I enter an email that's already registered and click "Create Account"
- Then I should see an error message saying "This email is already registered"

Scenario: Accessing a protected page without logging in

- Given I'm not logged in
- When I try to visit the dashboard URL directly
- Then I should be redirected to the login page

This format is clear enough that anyone can follow it, and it's specific enough that you know exactly what to check. You can hand this list to a friend and they'll know exactly what to test.

Happy Path vs Edge Cases

There are two types of test scenarios, and most beginners only think about one of them.

The Happy Path

This is the ideal flow. Everything goes right. The user enters correct information, clicks the right buttons in the right order, has a stable internet connection, and uses a modern browser. Testing the happy path confirms that your core features work as intended.

Edge Cases

This is where things get interesting. Edge cases are the unusual, unexpected, or extreme situations that users will inevitably encounter. They're called "edge" cases because they happen at the edges of normal usage.

Category	Happy Path	Edge Cases
Form input	User enters a valid email	User enters "abc", or leaves it blank, or pastes a 500-character string
Authentication	User logs in with correct password	User enters wrong password 10 times, or tries to log in with an unverified account
File upload	User uploads a 2MB JPEG	User uploads a 50MB file, a .exe file, or nothing at all
Search	User searches for "running shoes"	User searches for "", or "a", or a search term with special characters like "shoes & boots"
Payment	User completes checkout successfully	User's card is declined, or they close the browser mid-payment, or they hit the back button rapidly, or opens the same page in multiple tabs
Navigation	User clicks through pages in order	

The happy path is important but it's the easy part. Edge cases are where most bugs live, and where most of your testing effort should be focused.

Testing Forms (The Most Common Source of Bugs - don't forget to use the form validation guide in the resources tab!)

Forms are where users give you data, and they're the number one source of bugs in most web apps. Here's a comprehensive list of form scenarios to test:

Empty fields:

- Submit the form with all fields empty
- Submit the form with only some fields filled in
- Fill in everything except the required field and submit

Invalid input:

- Email field: enter "notanemail", "user@", "@domain.com"
- Phone field: enter letters, special characters, too few or too many digits
- Number fields: enter negative numbers, decimals, extremely large numbers
- Date fields: enter dates in the past (if not allowed), dates far in the future

Boundary testing:

- Enter the minimum allowed characters (e.g., a 1-character name)
- Enter the maximum allowed characters (paste in a 10,000-character string)
- Enter text with special characters: < > & \" ' / \ ; --
- Enter text with emojis
- Enter text with Unicode characters (accented letters, Chinese characters, Arabic text)

Submission behaviour:

- Double-click the submit button rapidly (does it submit twice?)
- Click submit, then click it again while the first request is still processing

- Lose internet connection right as you click submit
- Submit the form, then hit the browser's back button and submit again

Here's a prompt that will generate form test scenarios for your specific app:

```
I have a registration form with these fields: email (required), password (required, min 8 characters), display name (required, max 50 characters), and an optional phone number. Generate 15 test scenarios including happy path, validation errors, boundary cases, and edge cases. Format each as Given/When/Then.
```

Testing Authentication Flows

Auth flows are critical because they guard everything behind them. If auth is broken, either nobody can get in (bad) or everyone can get in (much worse). Here are scenarios to test:

Login:

- Login with correct credentials
- Login with wrong password
- Login with an email that doesn't exist
- Login with correct credentials but unverified email
- Login and check that you're redirected to the right page
- Login, close the browser, reopen it, check if you're still logged in

Logout:

- Log out and verify you can't access protected pages
- Log out and check that the back button doesn't show the dashboard
- Log out on one tab while still using the app on another tab

Password reset:

- Request a password reset with a valid email
- Request a password reset with an email that doesn't exist (should not reveal whether the account exists)
- Click the reset link and set a new password
- Try to use the reset link twice (it should expire after first use)
- Try to use a reset link that's older than the expiry time

Session management:

- Leave the app open for an extended period (does the session expire gracefully?)

- Open the app in two different browsers with the same account
- Change your password in one browser and check what happens in the other

I'm using Supabase Auth with email/password login, magic links, and Google OAuth. Generate a comprehensive test checklist for all authentication flows including login, logout, password reset, magic link, OAuth, session expiry, and protected route access. Include both happy paths and edge cases.

Testing on Different Browsers

Your app might look perfect in Chrome and completely broken in Safari. Browser compatibility issues are more common than you'd think, especially with newer CSS features.

At minimum, test on these browsers:

Browser	Why It Matters	Common Issues
Chrome	Most popular desktop browser (~65% market share)	Your primary dev browser, so usually works
Safari	Default on all iPhones and Macs (~19% market share)	Date inputs, flexbox gaps, backdrop-filter
Firefox	Privacy-focused users (~3% market share)	Generally good, minor CSS differences
Edge	Default on Windows (~5% market share)	Chromium-based, so usually matches Chrome
Safari Mobile	Every iPhone user	Different viewport handling, smooth scrolling issues
Chrome Mobile	Most Android users	Generally good, but check touch interactions

You don't need to install every browser. For Safari on Windows, use BrowserStack (free trial) or ask someone with a Mac to check. For mobile testing, use your own phone plus browser DevTools' device emulation.

The most common cross-browser issues:

- Flexbox gap property: Older Safari versions don't support it
- Date input styling: Safari renders date inputs very differently from Chrome
- Smooth scrolling: scroll-behavior: smooth doesn't work in all browsers
- CSS backdrop-filter: Safari needs the `-webkit-` prefix
- Form autofill styling: Each browser styles autofilled inputs differently

Review my CSS and Tailwind classes for any properties that might not be supported in Safari or older browsers. Suggest fallbacks or alternatives for any compatibility issues you find.

Testing with Slow Connections

Not everyone has fibre internet. Some of your users might be on a spotty mobile connection, on a train going through a tunnel, or in a country with slower average speeds. Testing on a slow connection reveals issues you'd never see on your fast development machine.

How to simulate slow connections:

1. Open Chrome DevTools (F12)
2. Go to the Network tab
3. Find the throttling dropdown (it says "No throttling" by default)
4. Select "Slow 3G" or "Fast 3G"
5. Reload your page and watch what happens

What to check:

- How long does the initial page load take? (Aim for under 3 seconds on Fast 3G)
- Do images load in a reasonable time? Are they lazy-loaded?
- Do loading states appear while content is being fetched? Or does the user stare at a blank screen?
- What happens if an API call times out? Does the user see a helpful error or just a spinner that never stops?
- Can the user still interact with the page while background content is loading?

Common problems revealed by slow connection testing:

- No loading indicators (user thinks the app is broken)
- Huge images that take 10+ seconds to load
- API calls with no timeout or error handling
- Layout shifts as content loads in (things jumping around on the page)
- Buttons that can be clicked before the page is fully interactive

```
Audit my app for slow connection performance. Check for: missing
loading states, images without lazy loading, API calls without
timeout handling, and any components that would cause layout
shift while loading. Suggest improvements for each issue.
```

Accessibility Testing

We covered accessibility basics in the "Think Like a User" section, but here are specific test scenarios:

- Keyboard navigation: Can you Tab through every interactive element on the page in a logical order? Can you activate buttons with Enter? Can you close modals with Escape?

- Screen reader: Turn on VoiceOver (Mac: Cmd+F5) or NVDA (Windows, free) and navigate your app. Does it make sense when read aloud? Are images described? Are form labels announced?
- Zoom: Zoom your browser to 200% (Ctrl/Cmd + Plus). Does everything still look usable? Does text wrap properly? Do elements overlap?
- Colour blindness: Use Chrome DevTools > Rendering > "Emulate vision deficiencies" to see your app as someone with protanopia (red-blind) or deuteranopia (green-blind) would see it. Can you still distinguish all elements?
- Reduced motion: Some users set "Reduce Motion" in their OS settings. Do your animations respect prefers-reduced-motion?

Run an accessibility audit on my main page components. Check for: missing ARIA labels, improper heading hierarchy, form inputs without associated labels, images without alt text, insufficient colour contrast, and elements that can't be reached via keyboard. Provide specific fixes.

Creating a Test Checklist

A test checklist is simply a document that lists every scenario you need to verify before launching or deploying. You can create it in a Markdown file, a Google Sheet, or even a Notion page. The key is having a single place where all your test scenarios live.

Here's a template structure:

```
# Test Checklist - [App Name]

## Authentication
- [ ] Sign up with valid email and password
- [ ] Sign up with existing email (should show error)
- [ ] Login with correct credentials
- [ ] Login with wrong password
- [ ] Password reset flow
- [ ] Logout clears session
- [ ] Protected pages redirect to login

## Forms
- [ ] Submit with all valid data
- [ ] Submit with empty required fields
- [ ] Submit with invalid email format
- [ ] Double-click prevention on submit button
- [ ] Form shows loading state during submission
- [ ] Success message appears after submission
- [ ] Error messages are clear and specific

## Responsive Design
- [ ] Looks good at 375px (mobile)
- [ ] Looks good at 768px (tablet)
```

You can ask AI to generate a customised version:

```
I'm building a [describe your app] with these features: [list features]. Create a comprehensive test checklist in Markdown format covering: authentication, forms, responsive design, performance, accessibility, and cross-browser testing. Include both happy paths and edge cases for each section.
```

Using AI to Generate Test Scenarios

This is one of the areas where AI tools truly shine. AI is excellent at thinking about edge cases you'd never consider because it's been trained on millions of examples of things going wrong.

Here are prompts that will generate thorough test scenarios:

For a specific feature:

```
Generate 15 test scenarios for my checkout flow. The flow is: user selects a plan, enters payment details (Stripe), confirms purchase, and is redirected to the dashboard with their new subscription active. Include happy paths, validation errors, payment failures, and edge cases like browser back button, page refresh mid-checkout, and expired sessions.
```

For a complete app:

```
Here's my app structure: [describe pages and features]. Generate a complete test plan with scenarios grouped by feature area. For each scenario, specify: the precondition, the action, and the expected result. Focus especially on edge cases and error states that a first-time builder might miss.
```

For finding bugs you might have missed:

```
I'm about to launch my app. Here are the main user flows:
```

1. Sign up and verify email
2. Browse content and bookmark items
3. Upgrade to paid subscription
4. Update profile and preferences

```
What are the 10 most likely bugs I haven't thought of? For each, explain the scenario and what I should check.
```

For security-focused testing:

```
Review my app for security vulnerabilities a user might exploit.
Consider: SQL injection in search fields, XSS in user-generated
content, CSRF on state-changing requests, exposed API endpoints,
and rate limiting on login attempts. For each, describe a test
scenario I can manually verify.
```

In Cursor or Claude Code specifically:

```
Look at my API routes in /server/src/api/ and generate test
scenarios for each endpoint. Include: valid requests, invalid
payloads, missing authentication, and rate limiting scenarios.
Format as a Markdown checklist I can use during testing.
```

The "What If" Game

Here's a simple exercise that makes you better at testing: play the "What If" game with every feature you build.

For any feature, ask yourself:

- What if the user is not logged in?
- What if the user's session just expired?
- What if the internet drops right now?
- What if the user double-clicks that button?
- What if the user hits the back button?
- What if the user opens this in a new tab?
- What if the API returns an error?
- What if the API takes 30 seconds to respond?
- What if the user enters something I didn't expect?
- What if the user tries to do this on their phone?

Each "What if" is a potential test scenario. You won't catch everything, but playing this game for 5 minutes per feature will catch most of the obvious problems before your users do.

Practical Exercise: Test Your Most Important Feature

Pick the single most important feature in your app (probably sign-up or the core user action) and do the following:

1. Write 5 happy path scenarios (the ideal flow)
2. Write 10 edge case scenarios (things that could go wrong)

3. Test each scenario manually and note the result (pass/fail)
4. For any failures, ask your AI tool to fix the issue:

```
When I [describe what you did], the expected behaviour was  
[describe what should happen], but instead [describe what  
actually happened]. Here's the relevant code: [paste code].  
Fix this bug and add proper error handling so the user sees  
a helpful message instead of a broken state.
```

5. Re-test the failed scenarios after the fix

This cycle of write scenarios, test, fix, re-test is the core of quality assurance. It's not glamorous, but it's what separates apps that people trust from apps that people abandon.

Key Takeaways

- Testing is about finding where your app breaks, not proving it works. Assume there are bugs and go hunting for them.
- Write test scenarios in plain English. Given/When/Then format makes scenarios clear and executable by anyone, not just developers.
- Happy paths are the easy part. Edge cases are where bugs live. Spend most of your testing effort on unexpected inputs, error states, and unusual user behaviour.
- Forms are the number one source of bugs. Test empty fields, invalid inputs, boundary lengths, special characters, and double-submission for every form in your app.
- Authentication flows need thorough testing. Login, logout, password reset, session expiry, and protected route access all need to work perfectly because they guard everything else.
- Test on multiple browsers. Chrome and Safari are the minimum. Safari on iOS is particularly important since every iPhone user depends on it.
- Slow connection testing reveals hidden problems. Use Chrome DevTools' network throttling to simulate 3G speeds and check for missing loading states and timeout handling.
- A test checklist keeps you organised. Create a Markdown file with every scenario grouped by feature area. Check it off before every deployment.
- AI is exceptional at generating test scenarios. Ask it to find edge cases for your specific features. It will think of things you never would because it's seen millions of bugs before.
- Play the "What If" game. For every feature, spend 5 minutes asking "What if...?" questions. Each answer is a test scenario worth checking.